SSHHole | OpenSSH password authentication bypass

KieSec · Technical write-up · CVE-2026-123158 · 8 November 2026 (Guyana)

What I reproduced

I reproduced an OpenSSH password authentication bypass in an authorized lab environment. I enumerated SSH, tested an input one byte above musl's SHA-crypt limit, and reached an interactive shell as a service account configured without a usable password. In my captured connection, the first password prompt ended in a denial. The second submission of the same input opened the shell. I ran `whoami` to confirm the remote identity.

Scope. I used a standard SSH client against a deliberately vulnerable Alpine Linux server configured for the test. I used the public disclosure to understand why the observed behavior was possible.

Discovery credit. Özgün Kültekin of Wakefield discovered and reported the vulnerability. His research helped me trace how the server turned a hashing failure into an accepted login. Researcher profile · Original SSHHole disclosure · OpenSSH release notes

1. Confirming the SSH entry point

My Nmap scan used service detection and default scripts on TCP port 22. It reported `22/tcp open`, `OpenSSH 10.6`, and SSH protocol 2.0. The banner gave me a service and a version to investigate. Confirming exposure required more context about the build, authentication path, account state, and package revision.

```
                          nmap -sV -sC -p 22 $target
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-11-09 01:27 UTC
Nmap scan report [redacted]
Host is up (0.00070s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.6 (protocol 2.0)
```

Figure 1. My Nmap scan of the lab target. Host identifiers are masked.

2. Why a password that should fail passed

A normal Linux password field in `/etc/shadow` can contain a hash setting and its result. OpenSSH's direct password verifier uses that setting to recompute the hash of the submitted password and compare it with the stored field. A field containing exactly `*` marks an account with no usable password hash. It can still be eligible for SSH through another allowed method, such as public-key authentication. Original disclosure · Alpine advisory

Stage	What happens in the vulnerable path
Pick a hash setting	Because `*` is too short to supply a usable setting, OpenSSH takes a fallback setting from another local account. This extra work helps reduce username-enumeration timing differences.
Hit the length limit	When the selected setting invokes musl's SHA-256-crypt (`$5$`) or SHA-512-crypt (`$6$`), input longer than 256 bytes makes hashing fail. In this case, musl's `crypt()` returns a literal `*` as its error marker.
Compare strings	The vulnerable non-PAM check compares the returned string with the target account's stored field. The two strings are both `*`. The equality check accepts the password even though hashing failed.

My input was 257 copies of the ASCII character `A`, for 257 bytes. The published range for this SHA-crypt path is 257 to 1,024 bytes. At 256 bytes, the input is still within musl's SHA-crypt limit. Above 1,024 bytes, OpenSSH's own password length check stops the attempt. Alpine advisory · OpenSSH 10.6p1 authentication source

Why the second prompt mattered in my run

The public analysis explains the result I saw. On the first request, OpenSSH looks up a fallback setting. Those lookups can reuse musl's static shadow buffer and overwrite the target field the verifier is about to compare. On the second request in the same connection, OpenSSH already has the fallback setting cached. The verifier reads the target field again, so the returned error marker can now compare equal to the freshly read `*`. Original disclosure · Researcher's reproduction notes

What has to line up

- The target must run an unpatched OpenSSH server on musl-based Linux, using its direct password verifier with PAM disabled or unavailable.
- SSH password authentication must be enabled, and the connecting client must be able to reach the server.
- The attacker needs the name of an SSH-eligible account with a password field of exactly `*`. The selected fallback setting must drive the vulnerable SHA-crypt path, supplied here by another local account's `$5$` or `$6$` hash.

Alpine advisory · Wakefield research note

3. What my terminal captured

I used an ordinary SSH connection and submitted the same 257-byte test input at both prompts. The first attempt was denied. After the second prompt, I reached the target's shell and ran `whoami` to confirm I was operating as the intended service account.

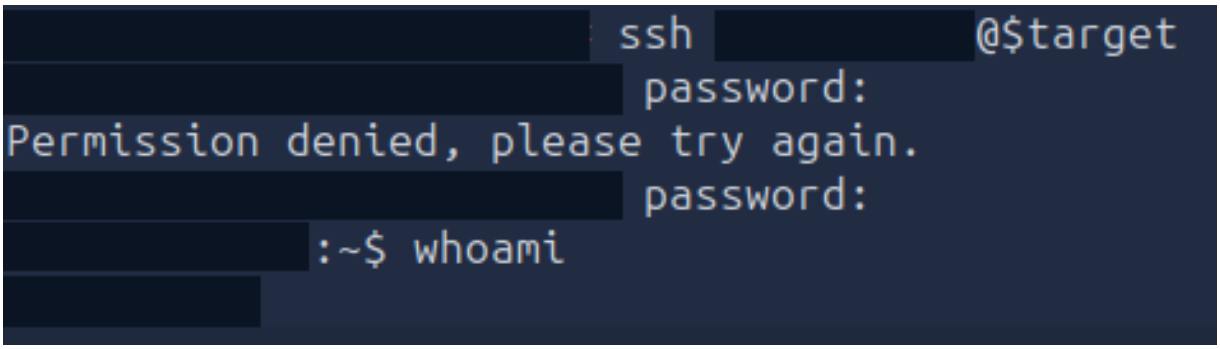


Figure 2. Screenshot from my SSH session.

What the foothold means

A successful bypass gives the attacker the target account's effective access. A service account might be able to read application data, change deployment files, or use credentials it can access. A permissive passwordless `doas` rule can turn that foothold into root. This is a separate privilege decision after login. Alpine documents that chain for certain official cloud images once a suitable donor hash is present. The terminal capture above establishes service-account access. Alpine advisory

4. Remediation and verification

Patch the server and restart the daemon. Upstream addressed the bypass in Portable OpenSSH 10.6p2 on 5 November 2026. The relevant hash-match path now requires a stored field of at least four bytes before it accepts a comparison. The patch also copies the shadow field into separately allocated memory, so later lookups cannot overwrite that copy. Alpine backported the fix. For example, 3.24 is fixed from `10.3_p1-r2` and 3.23 from `10.2_p1-r1`. Use the package revision for the deployed branch and restart the running `sshd` after upgrading. Release notes · Patch diff · Alpine fixed packages

Contain exposure while patching. Verify that a separate key-based admin login works and keep the existing admin session open. Set the effective `PasswordAuthentication no`, validate the configuration with `sshd -t`, and restart `sshd` using the system's service manager. Review included files and applicable `Match` blocks. Appending a second global directive can leave the earlier value in effect. Alpine advisory · sshd_config manual

Verify from a fresh connection. Check effective settings with `sshd -T`, using `-C` for the applicable user and connection context when `Match` rules apply. If password authentication was disabled, confirm the server no longer offers it for that account and that legitimate key access works. In an isolated patched test with password authentication still enabled, replay the previous test and confirm the account never opens a shell. These are verification steps for a follow-up test. The screenshots document the pre-fix session. sshd manual · Alpine advisory

Review signs of prior access. Look for `Accepted password for` events involving accounts intended for key-only access. Correlate the login with privilege elevation, new accounts, and changes to authorized SSH keys. A successful password login on one of these accounts is a stronger signal than a failed prompt alone. Alpine investigation guidance

References

- Özgün Kültekin's SSHHole disclosure and Wakefield research note
- Portable OpenSSH 10.6p2 release notes and upstream fix
- Alpine Linux security advisory and fixed versions
- CVSS 3.1 **8.1 High** (CNA). CVE-2026-123158 record